ALIANZA REBELDE

Guía completa del JavaScript

Para la exposición y defensa del proyecto

Qué hace cada función · Cómo modificarla · Arrow functions en JS y Java

Índice de contenidos

- 1. Estructura general del JS
- 2. Variables globales y localStorage
- 3. El evento DOMContentLoaded
- 4. SECCIÓN 1 — Hangar de Naves
- 4.1 cargarNaves()
- 4.2 cargarFiltroTipos()
- 4.3 aplicarFiltros()
- 4.4 ordenarNaves()
- 4.5 actualizarContador()
- 5. SECCIÓN 2 — Registro de Pilotos
- 5.1 guardarPiloto()
- 5.2 mostrarPilotos()
- 5.3 cargarSelectNaves()
- 5.4 editarPiloto() y eliminarPiloto()
- 5.5 guardarDatos() y cargarPilotos()
- 6. SECCIÓN 3 — Tablero de Misiones (Kanban)
- 6.1 añadirMision()
- 6.2 cargarDificultades()
- 6.3 pintarKanban()
- 6.4 moverMision() y eliminarMision()
- 6.5 filtrarMisiones()
- 7. SECCIÓN 4 — Dashboard
- 8. Guía de supervivencia: ¿Cómo tocar el código?
- 9. Arrow functions: JS vs Java — explicación completa

1. Estructura general del JavaScript

El archivo main.js está dividido en cuatro grandes bloques, uno por cada sección de la web. Todas las funciones se llaman desde un único punto de entrada llamado DOMContentLoaded, que garantiza que el HTML existe antes de intentar manipularlo.

Bloque	Qué controla
Variables globales	Datos en memoria (naves, pilotos, misiones) y variables de control
DOMContentLoaded	Arranca todo: llama las funciones iniciales y registra los eventos
Sección 1 — Hangar	Mostrar, filtrar y ordenar las tarjetas de naves
Sección 2 — Pilotos	Formulario CRUD (Crear, Leer, Editar, Borrar) de pilotos
Sección 3 — Kanban	Misiones con columnas To Do / In Progress / Done
Sección 4 — Dashboard	Estadísticas y barra de progreso calculadas en tiempo real

2. Variables globales y localStorage

Al principio del archivo se declaran cuatro variables que actúan como la 'memoria viva' de la aplicación:

```
let pilotos  = JSON.parse(localStorage.getItem('pilotos'))  || [];
let misiones = JSON.parse(localStorage.getItem('misiones')) || [];
let editIndex = null;
let estados  = ['todo', 'in_progress', 'done'];
```

Línea por línea:

Código	Qué significa
localStorage.getItem('pilotos')	Lee del almacenamiento del navegador el texto guardado con clave 'pilotos'
JSON.parse(...)	Convierte ese texto (formato JSON) de vuelta a un array de objetos JavaScript
|| []	Si no hay nada guardado (null), usa un array vacío en su lugar
editIndex = null	null significa 'no estamos editando nada'. Cuando el usuario pulsa Editar, guardamos aquí el número de fila
estados = [...]	Array fijo con los tres estados del kanban, en orden. Se usa para mover tarjetas

💡 **¿Qué es localStorage?**

Es una pequeña base de datos que el navegador guarda en tu ordenador. Los datos persisten aunque cierres la pestaña. JSON.stringify convierte el array a texto para guardarlo; JSON.parse lo convierte de vuelta al abrir la página.

3. El evento DOMContentLoaded

Todo el código que toca el HTML está envuelto en este evento:

```
document.addEventListener('DOMContentLoaded', () => {
    cargarNaves();
    mostrarPilotos();
    // ... más llamadas
    document.getElementById('btnAñadirMision')
        .addEventListener('click', añadirMision);
});
```

¿Por qué es necesario? El navegador lee el HTML de arriba a abajo. El <script> está casi al final del body, así que el HTML ya existe cuando se ejecuta. Pero si alguien moviera el script al <head>, el código fallaría sin este evento porque los botones y divs todavía no existirían en ese momento. Es una buena práctica tenerlo siempre.

Dentro también se configura la navegación entre secciones con un patrón muy limpio:

```
const secciones = {
    navButton1: 'Seccion1',
    navButton2: 'Seccion2',
    navButton3: 'Seccion3',
    navButton4: 'Seccion4'
};

Object.entries(secciones).forEach(([botonId, seccionId]) => {
    document.getElementById(botonId).addEventListener('click', () => {
        mostrarSeccion(seccionId);
    });
});
```

Object.entries convierte el objeto en un array de pares [clave, valor]. Luego forEach recorre ese array y para cada par asigna el evento click al botón correspondiente. Así con 4 líneas se gestionan los 4 botones del nav en vez de repetir código 4 veces.

4. Sección 1 — Hangar de Naves

4.1 cargarNaves()

Esta función vacía el contenedor de naves y lo vuelve a rellenar con los datos del array naves. Es el núcleo de la sección 1: cada vez que algo cambia (se ordena, se filtra) se llama esta función.

```
function cargarNaves() {
    const hangar = document.getElementById('contenedorNaves');
    hangar.innerHTML = '';                    // vaciar

    naves.forEach(nave => {                  // para cada nave...
        const card = document.createElement('div');
        card.classList.add('card');
        card.dataset.tipo = nave.tipo;        // guarda el tipo en el HTML

        card.innerHTML = `
            <h3>${nave.nombre}</h3>
            <p>Tipo: ${nave.tipo}</p>
            ...`,
        hangar.appendChild(card);            // añade al DOM
    });

    actualizarContador();
    cargarFiltroTipos();
}
```

Puntos clave: innerHTML = '' limpia el contenedor antes de repintarlo. card.dataset.tipo guarda el tipo de nave directamente en el elemento HTML como atributo data-tipo, y luego aplicarFiltros() lo lee para filtrar sin consultar el array.

4.2 cargarFiltroTipos()

Rellena el <select> de tipos de nave con los valores únicos que hay en el array, para que si añades más naves en el futuro el filtro se actualice solo.

```
const tiposUnicos = [...new Set(naves.map(n => n.tipo))];
```

Esta línea hace tres cosas encadenadas:

Parte	Qué hace
naves.map(n => n.tipo)	Crea un array con solo los tipos: ['caza', 'transporte', 'caza']
new Set(...)	Un Set es como un array pero sin duplicados: {'caza', 'transporte'}
[...Set]	El spread (...) convierte el Set de vuelta a array para poder usar forEach

4.3 aplicarFiltros()

Esta función es el motor de búsqueda. Combina el texto del buscador y el tipo seleccionado para mostrar u ocultar tarjetas.

```
function aplicarFiltros() {
    const texto = document.getElementById('buscador').value.toLowerCase();
    const tipo  = document.getElementById('filtroTipo').value;

    const cards = document.querySelectorAll('.card');

    cards.forEach(card => {
        const nombre = card.querySelector('h3').textContent.toLowerCase();
        const coincideNombre = nombre.includes(texto);
        const coincideTipo   = tipo === 'Todos' || card.dataset.tipo === tipo;

        if (coincideNombre && coincideTipo) {
            card.classList.remove('invisible');
        } else {
            card.classList.add('invisible');
        }
    });
    actualizarContador();
}
```

No borra ni recrea ningún elemento. Solo añade o quita la clase CSS 'invisible' (que tiene display:none) dependiendo de si la tarjeta pasa el filtro. Así es mucho más rápido que repintar todo el DOM.

4.4 ordenarNaves()

```
let ordenAsc = true;

function ordenarNaves() {
    if (ordenAsc) {
        naves.sort((a, b) => a.velocidad - b.velocidad);  // ascendente
    } else {
        naves.sort((a, b) => b.velocidad - a.velocidad);  // descendente
    }
    ordenAsc = !ordenAsc;   // alterna para la próxima vez
    cargarNaves();
}
```

sort() ordena el array en su sitio (lo modifica directamente). La función comparadora (a, b) => a.velocidad - b.velocidad funciona así: si el resultado es negativo, 'a' va antes; si es positivo, 'b' va antes. Invertir los operandos invierte el orden.

4.5 actualizarContador()

```
function actualizarContador() {
    const cards = document.querySelectorAll('.card');
    const visibles = [...cards].filter(c
=> !c.classList.contains('invisible')).length;
    document.getElementById('contador').textContent = `Mostrando ${visibles}
naves`;
}
```

querySelectorAll devuelve un NodeList, no un array. El spread [...cards] lo convierte a array para poder usar filter(). Luego se cuentan las que no tienen la clase 'invisible'.

5. Sección 2 — Registro de Pilotos

5.1 guardarPiloto(e)

Se activa cuando el usuario envía el formulario. La 'e' que recibe como parámetro es el objeto del evento submit.

```
function guardarPiloto(e) {
    e.preventDefault();   // CRÍTICO: evita que la página se recargue

    const piloto = {
        nombre:   document.getElementById('nombre').value,
        rango:    document.getElementById('rango').value,
        nave:     document.getElementById('nave').value,
        victorias: parseInt(document.getElementById('victorias').value),
        estado:   document.getElementById('estado').value
    };

    if (editIndex === null) {
        pilotos.push(piloto);        // modo CREAR
    } else {
        pilotos[editIndex] = piloto; // modo EDITAR
        editIndex = null;
    }

    guardarDatos();
    mostrarPilotos();
    cargarPilotos();
    actualizarDashboard();
}
```

parseInt() convierte el texto del input a número entero. Sin esto, las victorias serían texto ('5') y al comparar en el dashboard el mayor tendría resultados incorrectos. El patrón editIndex === null para distinguir crear/editar es un truco habitual: null actúa como bandera.

5.2 mostrarPilotos()

Reconstruye la tabla de pilotos desde cero. Cada fila incluye botones con onclick que llaman a funciones pasándoles el índice del piloto como argumento.

```
pilotos.forEach((p, i) => {
    const fila = document.createElement('tr');
    fila.innerHTML = `
        <td>${p.nombre}</td>
        ...
        <td>
            <button onclick='editarPiloto(${i})'>Editar</button>
            <button onclick='eliminarPiloto(${i})'>Eliminar</button>
        </td>
    `;
    lista.appendChild(fila);
});
```

El i en forEach es el índice del elemento. Se inyecta directamente en el onclick del HTML generado. Cuando el usuario pulsa el botón, el navegador ejecuta eliminarPiloto(0) o eliminarPiloto(2), etc.

5.3 cargarSelectNaves()

Rellena el desplegable de naves del formulario de pilotos. Se llama una vez al arrancar.

```
function cargarSelectNaves() {
    const select = document.getElementById('nave');
    select.innerHTML = `<option value=''>Selecciona nave</option>`;

    naves.forEach(n => {
        const option = document.createElement('option');
        option.value = n.nombre;
        option.textContent = n.nombre;
        select.appendChild(option);
    });
}
```

5.4 editarPiloto() y eliminarPiloto()

```
function editarPiloto(i) {
    const p = pilotos[i];
    document.getElementById('nombre').value    = p.nombre;
    document.getElementById('victorias').value = p.victorias;
    // ... rellena todos los campos
```

```
    editIndex = i;   // activa el modo edición
}

function eliminarPiloto(i) {
    if (!confirm('¿Seguro?')) return;  // pide confirmación
    pilotos.splice(i, 1);              // borra 1 elemento en posición i
    guardarDatos();
    mostrarPilotos();
}
```

splice(i, 1) modifica el array eliminando exactamente 1 elemento en el índice i. Los elementos posteriores se desplazan automáticamente.

5.5 guardarDatos() y cargarPilotos()

```
function guardarDatos() {
    localStorage.setItem('pilotos', JSON.stringify(pilotos));
}

function cargarPilotos() {
    const select = document.getElementById('selectPiloto');
    select.innerHTML = `<option value='default'>Selecciona piloto</option>`;
    pilotos.forEach(p => {
        const option = document.createElement('option');
        option.value = p.nombre;
        option.textContent = p.nombre;
        select.appendChild(option);
    });
}
```

guardarDatos convierte el array de pilotos a texto JSON y lo guarda en el navegador. cargarPilotos rellena el select de misiones para que el usuario pueda asignar un piloto a cada misión.

6. Sección 3 — Tablero de Misiones (Kanban)

6.1 añadirMision()

```
function añadirMision() {
    const mision = {
        id:          Date.now(),   // número único basado en la hora actual
        nombre:      document.getElementById('nombreMision').value,
        descripcion: document.getElementById('descripcionMision').value,
        piloto:      document.getElementById('selectPiloto').value,
        dificultad:  document.getElementById('selectDificultad').value,
        estado:      'todo'        // siempre empieza en la primera columna
```

```
    };

    if (!mision.nombre || mision.dificultad === 'default') return;

    misiones.push(mision);
    guardarMisiones();
    pintarKanban();
    actualizarDashboard();
}
```

Date.now() devuelve los milisegundos desde el 1 de enero de 1970. Es prácticamente imposible que dos misiones se creen en el mismo milisegundo, así que funciona como ID único. Se usa en moverMision y eliminarMision para encontrar la misión correcta sin depender de su posición en el array.

6.2 cargarDificultades()

```
function cargarDificultades() {
    const dificultad = ['facil', 'media', 'dificil', 'suicida'];

    dificultad.forEach(d => {
        const op1 = document.createElement('option');
        op1.value = d;
        op1.textContent = d;

        const op2 = op1.cloneNode(true);  // copia exacta del elemento

        select.appendChild(op1);           // para añadir misión
        filtro.appendChild(op2);           // para filtrar
    });
}
```

cloneNode(true) clona un elemento del DOM incluyendo todos sus hijos y atributos. El true indica que es una copia profunda (deep clone). Así no hay que crear el option dos veces para los dos selects.

6.3 pintarKanban()

Esta función acepta un parámetro opcional lista. Si no se le pasa nada, usa el array completo de misiones (comportamiento por defecto). Esto permite al filtro pasarle un subconjunto sin modificar el array original.

```
function pintarKanban(lista = misiones) {   // valor por defecto
    document.querySelectorAll('.columna').forEach(c => c.innerHTML = '');

    lista.forEach(m => {
        const card = document.createElement('div');
```

```
        card.innerHTML = `
            <h4>${m.nombre}</h4>
            <button onclick='moverMision(${m.id}, -1)'>⬅</button>
            <button onclick='moverMision(${m.id},  1)'>➡</button>
            <button onclick='eliminarMision(${m.id})'>X</button>
        `;
        const col = document.querySelector(`[data-estado='${m.estado}']`);
        if (col) col.appendChild(card);
    });
}
```

document.querySelector('[data-estado="todo"]') selecciona el div cuyo atributo data-estado sea 'todo'. Así cada tarjeta va a parar a la columna correcta usando el propio HTML como guía.

6.4 moverMision() y eliminarMision()

```
function moverMision(id, dir) {
    const m = misiones.find(x => x.id === id);  // busca por id
    if (!m) return;

    let i = estados.indexOf(m.estado);  // índice actual: 0, 1 o 2
    i += dir;                           // dir es -1 o +1

    if (i >= 0 && i < estados.length) {
        m.estado = estados[i];          // asigna el nuevo estado
    }

    guardarMisiones();
    pintarKanban();
    actualizarDashboard();
}

function eliminarMision(id) {
    misiones = misiones.filter(m => m.id !== id);  // descarta la que coincide
    guardarMisiones();
    pintarKanban();
    actualizarDashboard();
}
```

find() devuelve el primer elemento que cumple la condición (el que tiene ese id). indexOf() devuelve la posición de un valor en el array. Sumando dir (+1 o -1) se avanza o retrocede entre estados. El if evita salirse de los límites del array.

filter() en eliminarMision crea un nuevo array excluyendo la misión con ese id. A diferencia de splice, no modifica el array original sino que devuelve uno nuevo, que se reasigna a la variable misiones.

6.5 filtrarMisiones(e)

```
function filtrarMisiones(e) {
    const valor = e.target.value;   // valor seleccionado en el select

    if (valor === 'default') {
        pintarKanban();             // sin filtro: muestra todo
    } else {
        pintarKanban(misiones.filter(m => m.dificultad === valor));
    }
}
```

e.target es el elemento HTML que disparó el evento (el <select>). e.target.value es el valor seleccionado. filter() crea un array temporal con solo las misiones que coinciden y se lo pasa a pintarKanban, sin tocar el array original.

7. Sección 4 — Dashboard

actualizarDashboard() es la función más larga pero conceptualmente sencilla: filtra los arrays para contar elementos según su estado y los muestra en los párrafos del HTML. Lo más interesante es el uso de reduce():

```
// Encontrar el piloto con más victorias
const mejor = pilotos.reduce((max, p) => {
    if (p.victorias > max.victorias) {
        return p;    // p pasa a ser el nuevo máximo
    } else {
        return max;  // max sigue siendo el máximo
    }
});
```

reduce() recorre el array acumulando un resultado. Aquí el acumulador (max) empieza siendo el primer elemento, y en cada paso se compara con el siguiente. Si el siguiente tiene más victorias, se convierte en el nuevo max. Al final, max es el piloto con más victorias.

```
// Barra de progreso
const total = misiones.length;
const completadas = misiones.filter(m => m.estado === 'done').length;
let porcentaje = total > 0 ? (completadas / total) * 100 : 0;

barra.style.width   = porcentaje + '%';
barra.textContent   = Math.round(porcentaje) + '%';
```

El operador ternario (condición ? siVerdadero : siFalso) evita dividir entre cero cuando no hay misiones. barra.style.width cambia el ancho CSS directamente desde JS, lo que activa la animación de transición definida en el CSS.

8. Guía de supervivencia: ¿Cómo tocar el código?

Esta sección responde a la pregunta que más miedo da: '¿Y si la profesora te pide cambiar algo en vivo?'

Añadir una nave nueva al hangar

```
// Busca el array naves al principio del archivo y añade un objeto:
{
    imagen: null,
    nombre: 'A-Wing',
    tipo: 'caza',
    velocidad: 1300,
    tripulacion: 1,
    estado: 'operativa'
}
```

Solo hay que añadir el objeto al array. cargarNaves() lo pintará automáticamente, cargarFiltroTipos() actualizará el filtro y el dashboard contará la nueva nave.

Añadir un campo nuevo al formulario de pilotos (ej: especialidad)

Son tres pasos:

Paso 1 — En el HTML, añadir dentro del <form id='formPiloto'>:

```
<input type='text' id='especialidad' placeholder='Especialidad' required>
```

Paso 2 — En guardarPiloto(), añadir al objeto piloto:

```
especialidad: document.getElementById('especialidad').value,
```

Paso 3 — En mostrarPilotos(), añadir una celda <td> en el innerHTML de la fila y un <th> en el <thead> del HTML.

Añadir un nuevo estado al kanban (ej: 'blocked')

Son dos pasos:

Paso 1 — En el HTML añadir una nueva columna dentro de #kanban:

```
<div>
    <h2>Blocked</h2>
    <div class='columna' data-estado='blocked'></div>
</div>
```

Paso 2 — En el array estados del JS, añadir el nuevo valor en la posición deseada:

```
let estados = ['todo', 'blocked', 'in_progress', 'done'];
```

Cambiar las dificultades de misión

```
// En cargarDificultades(), modifica el array:
```

```
const dificultad = ['trivial', 'normal', 'dificil', 'imposible'];
```

Automáticamente se actualizarán los dos selects (añadir misión y filtrar).

Hacer que el buscador filtre también por tipo

Ya lo hace. El buscador llama a filtrarPorNombre() que internamente llama a aplicarFiltros(), que combina texto y tipo. Ambos filtros funcionan siempre juntos.

> 💡 **Regla de oro al modificar el código**
> Si cambias algo en el array de datos (naves, pilotos, misiones), generalmente no hay que tocar nada más: las funciones de pintado leen siempre el array entero. Si cambias la estructura de un objeto (añades un campo), hay que actualizar: (1) donde se crea el objeto, (2) donde se muestra en el HTML.

9. Arrow functions: JavaScript vs Java

Esto es lo que te provoca el cierre de culo cada vez que lo ves. Vamos a desmontarlo completamente con ejemplos de los dos lenguajes.

¿Qué es una arrow function / lambda?

Es una forma corta de escribir una función que se va a usar una sola vez, normalmente como argumento de otra función. En vez de darle nombre y escribirla aparte, se escribe directamente donde se necesita.

Equivalencia en JavaScript:

```
// Forma larga (función normal):
function sumar(a, b) {
    return a + b;
}

// Forma corta (arrow function), exactamente igual:
const sumar = (a, b) => a + b;
```

Sintaxis de arrow functions en JavaScript

Caso	Sintaxis
Sin parámetros	() => { ... }
Un parámetro	x => { ... } ó x => expresion
Varios parámetros	(a, b) => { ... }
Retorno inmediato (sin llaves)	(a, b) => a + b

Con cuerpo (varias líneas)	(a, b) => { let r = a+b; return r; }

Cuando no hay llaves, el resultado de la expresión se devuelve automáticamente (return implícito). Cuando hay llaves, hay que poner return explícitamente.

Arrow functions en tu código — ejemplos reales

forEach — hacer algo con cada elemento

```
// Recorre cada nave y ejecuta el bloque para cada una
naves.forEach(nave => {
    const card = document.createElement('div');
    // ... crea la tarjeta
});

// Sin arrow function, sería lo mismo:
naves.forEach(function(nave) {
    const card = document.createElement('div');
});
```

map — transformar cada elemento

```
// Crea un nuevo array con solo los tipos de nave
naves.map(n => n.tipo);
// Resultado: ['caza', 'transporte', 'caza']

// Equivalente sin arrow:
naves.map(function(n) { return n.tipo; });
```

filter — quedarse con los que cumplen una condición

```
// Pilotos activos
pilotos.filter(p => p.estado === 'activo');

// Misiones completadas
misiones.filter(m => m.estado === 'done');

// Equivalente sin arrow:
pilotos.filter(function(p) { return p.estado === 'activo'; });
```

find — encontrar el primero que cumple la condición

```
// Busca la misión con ese id
misiones.find(x => x.id === id);

// Devuelve el objeto mision, o undefined si no existe
```

sort — ordenar con criterio personalizado

```
// Ordena por velocidad ascendente
naves.sort((a, b) => a.velocidad - b.velocidad);

// El comparador recibe dos elementos (a, b) y devuelve:
//   número negativo → a va antes
//   número positivo → b va antes
//   0               → no cambia el orden
```

reduce — acumular un resultado

```
// Encuentra el piloto con más victorias
pilotos.reduce((max, p) => {
    if (p.victorias > max.victorias) return p;
    return max;
});

// reduce(callback) sin segundo arg usa el primer elemento como valor inicial
```

Lambdas en Java — misma idea, distinta sintaxis

En Java las lambdas llegaron en Java 8. La sintaxis es casi idéntica a JavaScript pero con algunas diferencias. El concepto es el mismo: función sin nombre que se pasa como argumento.

Sintaxis básica en Java

```
// (parámetros) -> cuerpo

// Sin parámetros:
() -> System.out.println("Hola")

// Un parámetro:
x -> x * 2

// Varios parámetros:
(a, b) -> a + b

// Con bloque de código:
(a, b) -> {
    int resultado = a + b;
    return resultado;
}
```

En JS la flecha es =>, en Java es ->. Aparte de eso, la lógica es idéntica.

Ejemplo 1: forEach en Java

```
List<String> nombres = Arrays.asList("Luke", "Leia", "Han");

// Con lambda:
nombres.forEach(nombre -> System.out.println(nombre));

// Sin lambda (forma antigua con clase anónima):
nombres.forEach(new Consumer<String>() {
    @Override
    public void accept(String nombre) {
        System.out.println(nombre);
    }
});
```

Ejemplo 2: filter en Java (con Stream)

```
List<Integer> numeros = Arrays.asList(1, 2, 3, 4, 5, 6);

// Quedarse solo con los pares:
List<Integer> pares = numeros.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
// Resultado: [2, 4, 6]
```

Ejemplo 3: sort en Java

```
List<String> pilotos = Arrays.asList("Han", "Luke", "Rey");

// Ordenar por longitud de nombre:
pilotos.sort((a, b) -> a.length() - b.length());
// Resultado: [Han, Rey, Luke]
```

Ejemplo 4: map en Java

```
List<String> nombres = Arrays.asList("luke", "leia", "han");

// Convertir a mayúsculas:
List<String> mayusculas = nombres.stream()
    .map(n -> n.toUpperCase())
    .collect(Collectors.toList());
// Resultado: [LUKE, LEIA, HAN]
```

Tabla comparativa JS vs Java

JavaScript	Java

naves.forEach(n => ...)	lista.forEach(n -> ...)
naves.map(n => n.tipo)	lista.stream().map(n -> n.getTipo()).collect(...)
pilotos.filter(p => p.estado === 'activo')	lista.stream().filter(p -> p.getEstado().equals("activo")).collect(...)
misiones.find(m => m.id === id)	lista.stream().filter(m -> m.getId() == id).findFirst().orElse(null)
naves.sort((a,b) => a.vel - b.vel)	lista.sort((a,b) -> a.getVel() - b.getVel())
arr.reduce((acc, x) => ...)	lista.stream().reduce(valorInicial, (acc, x) -> ...)
n => expresion (return implícito)	n -> expresion (return implícito)
(a,b) => { return ...; }	(a,b) -> { return ...; }

Cómo leer una lambda sin que te explote la cabeza

Cuando veas una lambda/arrow function, aplica esta lectura mental:

```
pilotos.filter(p => p.estado === 'activo')
```

```
// Lee así:
// 'De los pilotos, filtra. Para cada uno (llámalo p),
//  quedatelo si p.estado es igual a activo'
```

```
naves.sort((a, b) => a.velocidad - b.velocidad)
```

```
// Lee así:
// 'Ordena las naves. Para comparar dos (llámalas a y b),
//  resta sus velocidades (si negativo, a va antes)'
```

💡 **Resumen final para la exposición**
Las arrow functions (JS) y lambdas (Java) son lo mismo: funciones sin nombre que se escriben donde se usan. La flecha => en JS y -> en Java separan los parámetros del cuerpo. Sin llaves, el resultado se devuelve automáticamente. Con llaves, hay que escribir return. Se usan siempre con forEach, filter, map, sort, find y reduce.

— Que la Fuerza te acompañe en la exposición —